

Project Overview

Development of 'TasteTracker' PWA

Thomas Dubowski

Project Description

Brief

A local food blogger has requested development of a progressive web app (PWA) named '*Taste Tracker*'. The platform will allow users to share food reviews for locations in the central west.

Functional Requirements

- Users must have the ability to create accounts and sign-in in a secure way
- After logging in/creating account user can add restaurant reviews that include: ○ Restaurant name ○ Cuisine type ○ Review date ○ Rating ○ Text
- Non-logged in users have permission to see and search reviews, but cannot add reviews until logged in
- Users have ability to edit/delete reviews created by their account
- All users can search reviews and filter results

Non-Functional Requirements

- Clean, sleek and functional user interface (UI)
- App should function fully on mobile, desktop and tablet
- App should function offline using service worker
- Data must be stored securely in SQL database with appropriate encryption

Project Constraints

- Must use Python flask, Jinja2 templates, SQLite, HTML, CSS and JavaScript
- Must maintain up to date project version on github using commits Must use secure coding practices i.e. password hashing, block XSS, Authorisation checks, ect.

Acceptance Criteria

- Users can register, login, add/edit/delete reviews, and search/filter data
- Database stores all relevant review data
- Application code is well documented and modular
- Github repository reflect agile workflow with a project board

Project Planning

Data Dictionaries

Reviews Data Dictionary

Field Name	Data Type	Size	Key	Null	Description
review_id	integer		Primary	No	Unique identifier for each review in table
user_id	integer		Foreign	No	Identifier for the account that created the review
restaurant_name	varchar	100		Yes	Name of resturant
cuisine_type	varchar	50		No	Type of cuisine
rating	integer			Yes	Integer of 0-5
review_text	varchar	500		No	Text for review details
created_at	date			No	System generated review creation date

User Data Dictionary

Field Name	Data Type	Size	Key	Null	Description
user_id	integer		Primary	No	Unique Identifier for each user within the table
username	string	50	Unique	No	Unique name created by the user for login and identification
email	string	100		No	The users provided email on signup
password_hash	string	255		No	Password stored as hash for account authentication
created_at	date			No	System generated account creation date

IPO Charts

User Registration

Input	Process	Output
Username	User clicks submit button	Validation Success - Success message + redirect to login page
Email	Validate input (non-empty, unique email/username)	Validation failure - Error message
Password	Hash password using secure algorithm	
Submit button	Insert user data into Users table	

User Login

Input	Process	Output
Username	User clicks submit button	Validation Success - Member session active
Password	Validate input	Validation Failure - Error message
Submit button	Retrieve user from users table	
	Check hashed passwords	
	If credentials match, start session	

Add Review

Input	Process	Output
Restaurant name	User clicks submit button	Success popup
Cuisine	Validate inputs	New review appears in DB and on site
Rating	Insert into reviews table with user ID as FK	Failure - Error message
Review text		
Date		
Submit button		

Review Management

Input	Process	Output
Review ID	Review ID	Review ID
New/Replacement data (Edits)	New/Replacement data (Edits)	New/Replacement data (Edits)
Submit button	Submit button	Submit button

Search

Input	Process	Output
Keywords	User clicks submit button	Displayed list of reviews matching criteria
Cuisine type	Query review table with parameters	Failure - Error message
Rating	Sort results by rating, date, ect.	
Order	Present filtered list	
Submit button		

Storyboard

1. Home Page (All Users)

Purpose: Acts as a landing page when you first access the site

Features: Welcome message and site description

Navigation:

Home > Drop down menu > Search, Login, Register or All Reviews page

2. Register Page

Purpose: Allow new users to create an account

Features: Interactive form for account creation

Navigation:

Register > Submit button > Login page

Register > 'Already have an account?' hyperlink > Login page

Register > Drop Down menu > Search, Login, Register or All Reviews page

3. Login Page

Purpose: Allows users with an account to log in for their member features and personal reviews

Features: Interactive form for account login

Navigation:

Login > Login Button > User Dashboard

Login > Drop Down Menu > Search, Login, Register or All Reviews page

Login > Invalid login > Login and error message

4. Dashboard / User Home (Logged-in Users)

Purpose: Main page for logged in users

Features: Welcome message with username, member pages access

Navigation:

Dashboard > Drop Down Menu > Add Review, My Reviews, Search Reviews

Dashboard > Drop Down Menu > Logout > Homepage

5. Add Review Page

Purpose: Allow logged-in users to submit reviews.

Features: Review submission form, Rating system with interactive stars

Navigation:

Add Review > Submit button > My Reviews

Add Review > Drop Down Menu > Dashboard, Add Review, My Reviews, Search Reviews

Add Review > Drop Down Menu > Logout > Homepage

6. Edit Review Page

Purpose: Allows user to edit their own existing reviews

Features: Edit form, pre filled with original data, save and delete options

Navigation:

Edit Review > Save Changes Button > My Reviews

Edit Review > Delete Button > My Reviews

Edit Review > Drop Down Menu > Dashboard, Add Review, My Reviews, Search Reviews

Edit Review > Drop Down Menu > Logout > Homepage

7. Offline Page (PWA Feature)

Purpose: Alerts the user that they are offline

Features: Null

Pseudocode

FUNCTION generate_csrf_token():

IF '_csrf_token' not in session:

 CREATE random token

 SAVE to session

RETURN token

FUNCTION validate_csrf():

token = session token

form_token = form token

IF token missing OR mismatch:

 ABORT 403

FUNCTION get_current_user():

IF user_id in session:

 RETURN User object

ELSE:

 RETURN None

ROUTE '/' [GET]:

page = request page number

reviews = fetch paginated reviews ordered by date desc

RENDER 'base.html' with reviews

ROUTE '/search' [GET]:

query, cuisine, min_rating, sort = request args

reviews = Review table

APPLY filters:

- restaurant_name or review_text LIKE query

- cuisine_type == cuisine

- rating >= min_rating

APPLY sorting:

- if sort='rating' → order by rating desc

- else → order by date desc

RENDER 'search.html' with reviews

ROUTE '/login' [GET, POST]:

IF POST:

 validate_csrf()

 username, password = form

 user = find user by username

 IF user exists AND password correct:

 SET session

 REDIRECT home

 ELSE:

 return error 401

ELSE:

 RENDER 'login.html'

ROUTE '/logout' [GET]:

- CLEAR session
- REDIRECT home

ROUTE '/create_account' [GET, POST]:

IF POST:

- validate_csrf()
- CREATE new user with hashed password
- COMMIT to DB
- REDIRECT home

ELSE:

- RENDER 'create_account.html'

ROUTE '/add_review' [GET, POST]:

IF GET:

- RENDER 'add_review.html'

ELSE POST:

- validate_csrf()
- CHECK required fields & rating 1-5
- CREATE Review
- COMMIT
- flash success
- REDIRECT home

ROUTE '/edit_review/<review_id>' [GET, POST]:

CHECK login

review = fetch review

CHECK user owns review

IF POST:

- validate_csrf()
- UPDATE review fields
- COMMIT
- REDIRECT my_reviews

ELSE:

- RENDER 'edit_review.html' with review

ROUTE '/delete_review/<review_id>' [POST]:

validate_csrf()

CHECK login & ownership

DELETE review

COMMIT

flash success

REDIRECT my_reviews

ROUTE '/profile' [GET]:

CHECK login

user = get_current_user()

RENDER 'profile.html' with user

ROUTE '/profile/edit' [GET, POST]:

```
CHECK login
IF POST:
    validate_csrf()
    UPDATE username & email (check duplicates)
    COMMIT
    flash success
    REDIRECT profile
ELSE:
    RENDER 'edit_profile.html'
```

```
ROUTE '/profile/change_password' [POST]:
    validate_csrf()
    CHECK current password
    CHECK new passwords match
    UPDATE password_hash
    COMMIT
    flash success
    REDIRECT profile
```

```
ROUTE '/profile/delete' [POST]:
    validate_csrf()
    CHECK login
    DELETE user
    CLEAR session
    COMMIT
    flash success
    REDIRECT home
```